我想要基于 Netket 来计算分子的基态能量 我计划通过两种方式 第一种：基于原生 Netket 流程 效果非常好 精准求解

```
    #我要尝试一下 自己使用 FFN 来求解 H2分子的系统基态
import jax
import jax.numpy as jnp
import netket as nk
import netket.experimental as nkx
import numpy as np
from pyscf import gto, scf, fci
from flax import nnx
import optax
from tqdm import tqdm
from functools import partial

# ==============================================================================
# 1. 全局参数 & H₂ 分子定义
# ==============================================================================
bond_length = 1.4
geometry = [('H', (0., 0., 0.)), ('H', (bond_length, 0., 0.))]
mol = gto.M(atom=geometry, basis='STO-3G', verbose=0)
mf = scf.RHF(mol).run(verbose=0)

# FCI 精确基准
cisolver = fci.FCI(mf)
cisolver.nroots = 4
E_fcis, fcivec = cisolver.kernel()
print("="*60)
print("H₂ FCI 基准能量")
print("="*60)
for i, e in enumerate(E_fcis):
    exc = (e - E_fcis[0]) * 27.2114
    print(f"E{i} = {e:.8f} Ha  |  激发能: {exc:.4f} eV")

# NetKet 哈密顿量
ha = nkx.operator.from_pyscf_molecule(mol)

# 原始 Hilbert 空间（2个轨道，每个自旋1个电子）
hi = nk.hilbert.SpinOrbitalFermions(
    n_orbitals=2,
    s=1/2,
    n_fermions_per_spin=(1,1),
)

class SingleStateAnsatz(nnx.Module):
    def __init__(self, n_spin_orbitals: int, hidden_dim=16, *, rngs: nnx.Rngs):
        super().__init__()
        self.linear1 = nnx.Linear(n_spin_orbitals, hidden_dim, rngs=rngs, param_dtype=complex)
        self.linear2 = nnx.Linear(hidden_dim, hidden_dim, rngs=rngs, param_dtype=complex)
        self.output = nnx.Linear(hidden_dim, 1, rngs=rngs, param_dtype=complex)

    def __call__(self, x):
        h = nnx.tanh(self.linear1(x.astype(complex)))
        h = nnx.tanh(self.linear2(h))
        out = self.output(h)
        return jnp.squeeze(out)

edges = [(0, 1), (2, 3)]
g = nk.graph.Graph(edges=edges)
single_rule = nk.sampler.rules.FermionHopRule(hilbert=hi, graph=g)
sampler = nk.sampler.MetropolisSampler(hi, rule=single_rule, n_chains=16, sweep_size=32)

rngs = nnx.Rngs(21)
model = SingleStateAnsatz(n_spin_orbitals=4, hidden_dim=16, rngs=rngs)

vstate = nk.vqs.MCState(
    sampler=sampler,
    model=model,
    n_samples=512,
    n_discard_per_chain=10
```

```
)
preconditioner =nk.optimizer.SR(diag_shift=0.01)
optimizer = nk.optimizer.Sgd(0.01)
vmc = nk.driver.VMC(ha,optimizer,variational_state=vstate,preconditioner=preconditioner)
vmc.run(300)
```

第二种:基于自己写的方式运算 其中 sampler 等是 Netket的 效果非常差

```
import jax
from jax import flatten_util
import jax.numpy as jnp
import netket as nk
import netket.experimental as nkx
import numpy as np
from pyscf import gto, scf, fci
from flax import nnx
import optax
from tqdm import tqdm
from functools import partial

# ==============================================================================
# 1. 全局参数 & H₂ 分子定义
# ==============================================================================
bond_length = 1.4
geometry = [('H', (0., 0., 0.)), ('H', (bond_length, 0., 0.))]
mol = gto.M(atom=geometry, basis='STO-3G', verbose=0)
mf = scf.RHF(mol).run(verbose=0)

# FCI 精确基准
cisolver = fci.FCI(mf)
cisolver.nroots = 4
E_fcis, fcivec = cisolver.kernel()
print("="*60)
print("H₂ FCI 基准能量")
print("="*60)
for i, e in enumerate(E_fcis):
    exc = (e - E_fcis[0]) * 27.2114
    print(f"E{i} = {e:.8f} Ha  |  激发能: {exc:.4f} eV")

# NetKet 哈密顿量
ha = nkx.operator.from_pyscf_molecule(mol)

# 原始 Hilbert 空间（2个轨道，每个自旋1个电子）
hi = nk.hilbert.SpinOrbitalFermions(
    n_orbitals=2,
    s=1/2,
    n_fermions_per_spin=(1,1),
)

# ==============================================================================
# 2. 神经网络 Ansatz
# ==============================================================================
class SingleStateAnsatz(nnx.Module):
    def __init__(self, n_spin_orbitals: int, hidden_dim=16, *, rngs: nnx.Rngs):
        super().__init__()
        self.linear1 = nnx.Linear(n_spin_orbitals, hidden_dim, rngs=rngs, param_dtype=complex)
        self.linear2 = nnx.Linear(hidden_dim, hidden_dim, rngs=rngs, param_dtype=complex)
        self.output = nnx.Linear(hidden_dim, 1, rngs=rngs, param_dtype=complex)

    def __call__(self, x):
        h = nnx.tanh(self.linear1(x.astype(complex)))
        h = nnx.tanh(self.linear2(h))
        out = self.output(h)
        return jnp.squeeze(out)

def forward(GraphDef_State, x):
    log_psi, _ = nnx.call(GraphDef_State)(x)
    return log_psi
```

```python
@partial(jax.jit, static_argnames=("model_forward",))
def energy_and_grad(GraphDef_State, model_forward, hamiltonian, samples):
    graphdef, state = GraphDef_State

    def loss_fn(s):
        log_psi = model_forward((graphdef, s), samples)
        eta, H_sigmaeta = hamiltonian.get_conn_padded(samples)
        logpsi_eta = model_forward((graphdef, s), eta)

        log_psi = jnp.expand_dims(log_psi, -1)
        Eloc = jnp.sum(H_sigmaeta * jnp.exp(logpsi_eta - log_psi), axis=-1)
        energy = jnp.mean(Eloc)
        return energy.real, energy

    (loss, energy), grads = jax.value_and_grad(loss_fn, has_aux=True)(state)
    return energy, grads

# ==============================================================================
# 3. QGT & SR 核心函数
# ==============================================================================
@partial(jax.jit, static_argnums=0)
def compute_qgt_correct(forward_fn, GraphDef_State, samples):
    graphdef, state = GraphDef_State

    samples_flat = samples.reshape((-1, samples.shape[-1]))
    n_samples = samples_flat.shape[0]

    def logpsi_given_state(s, x):
        return forward_fn((graphdef, s), x)

    def grad_logpsi_single(x):
        return jax.grad(logpsi_given_state, argnums=0, holomorphic=True)(state, x)

    grads_tree = jax.vmap(grad_logpsi_single)(samples_flat)

    grads_flat, _ = jax.tree_util.tree_flatten(grads_tree)
    grads_concat = jnp.concatenate([g.reshape((n_samples, -1)) for g in grads_flat], axis=-1)

    mean_grad = jnp.mean(grads_concat, axis=0, keepdims=True)
    centered_grads = grads_concat - mean_grad

    qgt = (1.0 / n_samples) * jnp.einsum('si,sj->ij', centered_grads.conj(), centered_grads)
    qgt = qgt.real

    return qgt

def flatten_pytree(pytree):
    leaves, tree_def = jax.tree_util.tree_flatten(pytree)
    flat = jnp.concatenate([jnp.ravel(leaf) for leaf in leaves])
    return flat, tree_def, [leaf.shape for leaf in leaves]

def unflatten_pytree(flat_vec, tree_def, shapes):
    leaves = []
    idx = 0
    for shape in shapes:
        size = jnp.prod(jnp.array(shape))
        leaf = flat_vec[idx:idx+size].reshape(shape)
        leaves.append(leaf)
        idx += size
    return jax.tree_util.tree_unflatten(tree_def, leaves)
# 新增：导入 JAX 的展平工具
from jax import flatten_util

# ==============================================================================
# 修复版 SR 核心函数：使用 jax.flatten_util.ravel_pytree
# ==============================================================================
@partial(jax.jit, static_argnums=0)
def compute_natural_gradient(forward_fn, GraphDef_State, samples, E_grad, epsilon=1e-3):
    """
    修复版：使用 jax.flatten_util 安全展平/恢复参数树
    """
    # 1. 计算 QGT
    qgt = compute_qgt_correct(forward_fn, GraphDef_State, samples)
```

```python
    # 2. 正则化 QGT
    n_params = qgt.shape[0]
    qgt_reg = qgt + epsilon * jnp.eye(n_params)

    # ✅ 3. 关键修复：用 jax.flatten_util.ravel_pytree 展平梯度
    g_flat, unflatten_fn = flatten_util.ravel_pytree(E_grad)

    # 4. 解线性方程
    nat_g_flat = jnp.linalg.solve(qgt_reg, g_flat)

    # ✅ 5. 关键修复：用返回的 unflatten_fn 恢复梯度树
    nat_grad = unflatten_fn(nat_g_flat)

    return nat_grad, qgt

# ==============================================================================
# 4. 初始化 & 训练循环
# ==============================================================================
rngs = nnx.Rngs(21)
model = SingleStateAnsatz(4, 12, rngs=rngs)
GraphDef_State = nnx.split(model)  # ✅ 统一用这个变量名
graphdef, model_state = GraphDef_State

# 采样器初始化
edges = [(0, 1), (2, 3)]
g = nk.graph.Graph(edges=edges)
single_rule = nk.sampler.rules.FermionHopRule(hilbert=hi, graph=g)
sampler = nk.sampler.MetropolisSampler(hi, rule=single_rule, n_chains=16, sweep_size=32)

sampler_state = sampler.init_state(forward, GraphDef_State, seed=1)
samples, sampler_state = sampler.sample(forward, GraphDef_State, state=sampler_state, chain_length=200)
optimizer = optax.adam(learning_rate=0.01)
opt_state = optimizer.init(model_state)
n_iters = 300

# SR 超参数（最终最优配置）
use_sr = True
sr_interval = 1
sr_epsilon = 1e-4

for i in tqdm(range(n_iters)):
    # 1. 采样 + 【必修复】展平样本
    sampler_state = sampler.reset(forward, GraphDef_State, state=sampler_state)
    samples, sampler_state = sampler.sample(forward, GraphDef_State, state=sampler_state, chain_length=200)
    samples = samples.reshape(-1, 4)  # ✅ 修复1：展平为 (N,4)

    # 2. 计算能量和梯度
    E, E_grad = energy_and_grad(GraphDef_State, forward, ha, samples)

    # 3. SR 自然梯度
    if use_sr and (i % sr_interval == 0):
        nat_grad, qgt = compute_natural_gradient(
            forward, GraphDef_State, samples, E_grad, epsilon=sr_epsilon
        )
        update_grad = nat_grad

        if i % 50 == 0:
            print(f"\n{'='*60}")
            print(f"Iter {i:3d} | SR Enabled")
            print(f"Energy: {E.real:.8f} Ha  (FCI: {E_fcis[0]:.8f} Ha)")
            print(f"Error:  {abs(E.real - E_fcis[0]):.6f} Ha")
            print(f"QGT trace: {jnp.trace(qgt):.4f}")
            print(f"{'='*60}\n")
    else:
        update_grad = E_grad

    # 4. 参数更新
    updates, opt_state = optimizer.update(update_grad, opt_state, model_state)
    model_state = optax.apply_updates(model_state, updates)
    GraphDef_State = (graphdef, model_state)
```

```
    # 5. 打印
    if i % 10 == 0 and (i % 50 != 0):
        print(f"Iter {i:3d} | Energy: {E.real:.8f} Ha")
```

我需要警告你的是, 以下片段不可以更改 其次，要基于 flax.nnx

```
    #我要尝试一下 自己使用 FFN 来求解 H2分子的系统基态
import jax
import jax.numpy as jnp
import netket as nk
import netket.experimental as nkx
import numpy as np
from pyscf import gto, scf, fci
from flax import nnx
import optax
from tqdm import tqdm
from functools import partial

# ==============================================================================
# 1. 全局参数 & H₂ 分子定义
# ==============================================================================
bond_length = 1.4
geometry = [('H', (0., 0., 0.)), ('H', (bond_length, 0., 0.))]
mol = gto.M(atom=geometry, basis='STO-3G', verbose=0)
mf = scf.RHF(mol).run(verbose=0)

# FCI 精确基准
cisolver = fci.FCI(mf)
cisolver.nroots = 4
E_fcis, fcivec = cisolver.kernel()
print("="*60)
print("H₂ FCI 基准能量")
print("="*60)
for i, e in enumerate(E_fcis):
    exc = (e - E_fcis[0]) * 27.2114
    print(f"E{i} = {e:.8f} Ha  |  激发能: {exc:.4f} eV")

# NetKet 哈密顿量
ha = nkx.operator.from_pyscf_molecule(mol)

# 原始 Hilbert 空间 (2个轨道, 每个自旋1个电子)
hi = nk.hilbert.SpinOrbitalFermions(
    n_orbitals=2,
    s=1/2,
    n_fermions_per_spin=(1,1),
)

class SingleStateAnsatz(nnx.Module):
    def __init__(self, n_spin_orbitals: int, hidden_dim=16, *, rngs: nnx.Rngs):
        super().__init__()
        self.linear1 = nnx.Linear(n_spin_orbitals, hidden_dim, rngs=rngs, param_dtype=complex)
        self.linear2 = nnx.Linear(hidden_dim, hidden_dim, rngs=rngs, param_dtype=complex)
        self.output = nnx.Linear(hidden_dim, 1, rngs=rngs, param_dtype=complex)

    def __call__(self, x):
        h = nnx.tanh(self.linear1(x.astype(complex)))
        h = nnx.tanh(self.linear2(h))
        out = self.output(h)
        return jnp.squeeze(out)

edges = [(0, 1), (2, 3)]
g = nk.graph.Graph(edges=edges)
single_rule = nk.sampler.rules.FermionHopRule(hilbert=hi, graph=g)
sampler = nk.sampler.MetropolisSampler(hi, rule=single_rule, n_chains=16, sweep_size=32)
```

此外需要明确的是:Netket 的采样器接受的machine函数: machine (Union[Module, Callable[[Any, Union[ndarray, Array]], Union[ndarray, Array]]]) – A Flax module or callable with the forward pass of the log-pdf. If it is a callable, it should have the signature f(parameters, σ) -> jax.Array. 采样器返回的 shape=(n_chains, chain_length,hi.size)